

Lezione 14: Tipi Generici (Generics) e Classi Parametriche

Questo notebook raccoglie tutto il codice della Lezione 14 (`MavenDate`):

- `src/main/java/it/oop/core/Pair.java` (Coppia generica `Pair<F, S>`)
- `src/main/java/it/oop/core/OrderedPair.java` (Coppia ordinata con bound `T extends Comparable<T>`)
- `src/main/java/it/oop/core/DatePair.java` (`Pair<Date, Date>`)
- `src/main/java/it/oop/core/DateInterval.java` (`OrderedPair<Date>`)
- Tutte le classi `Date`, `FormattedDate`, `Time`, `TimeStamp`
- `src/main/java/it/oop/ui/MainDate.java`

Struttura dei file della lezione (path dalla cartella radice):

```
Programmazione-II/
├── codice-commentato/
│   └── Lezione14/
│       ├── MavenDate
│       │   ├── pom.xml
│       │   └── src
│       │       ├── main
│       │       │   ├── java
│       │       │   │   └── it
│       │       │   │       └── oop
│       │       │   │           ├── core
│       │       │   │           │   ├── AmericanDate.java
│       │       │   │           │   ├── Birthday.java
│       │       │   │           │   ├── Date.java
│       │       │   │           │   ├── DateInterval.java
│       │       │   │           │   ├── DatePair.java
│       │       │   │           │   ├── FormattedDate.java
│       │       │   │           │   ├── FormattedDateConverter.java
│       │       │   │           │   ├── ItalianDate.java
│       │       │   │           │   ├── OrderedPair.java
│       │       │   │           │   ├── Pair.java
│       │       │   │           │   ├── Time.java
│       │       │   │           │   └── TimeStamp.java
│       │       │   │           └── ui
│       │       │   │               ├── Date.java
│       │       │   │               └── MainDate.java
│       │       │   └── resources
│       │       └── test
│       │           ├── java
│       │           │   ├── it
│       │           │   │   └── oop
│       │           │   │       └── core
│       │           │   │           └── TestItalianDate.java
```

Argomenti trattati:

- Definizione di classi generiche con parametri di tipo (`<F, S>` , `<T>`)
- Bounded Type Parameters (`<T extends Comparable<T>>`) per vincolare i tipi accettabili
- Ereditarietà con classi generiche: specializzazione di `Pair` in `DatePair` e di `OrderedPair` in `DateInterval`
- Type safety a tempo di compilazione ed eliminazione di cast espliciti pericolosi

1. Interfaccia `Time`

```
In [1]: interface Time {
        int getSeconds();
        int getMinutes();
```

```
    int getHours();  
}
```

2. Classe `Date` con `Comparable<Date>`

```
In [2]: class Date implements Comparable<Date> {  
    protected int day;  
    protected int month;  
    protected int year;  
  
    public Date(int day, int month, int year) {  
        this.day = day;  
        this.month = month;  
        this.year = year;  
        verify();  
    }  
    public Date(int day, int month) {  
        this(day, month, 2025);  
    }  
    public Date(Date other) {  
        this.day = other.day;  
        this.month = other.month;  
        this.year = other.year;  
        verify();  
    }  
  
    void verify() {  
        if (year < 0 || month < 1 || month > 12)  
            System.out.println("Illegal date!"); // Illegal date!  
        else  
            if (day < 1 || day > daysPerMonth(month))  
                System.out.println("Illegal date!"); // Illegal date!  
    }  
  
    public int getDay() { return day; }  
    public int getMonth() { return month; }  
    public int getYear() { return year; }  
  
    public static int daysPerMonth(int month) {  
        int days;  
        switch(month) {  
            case 4:  
            case 6:  
            case 9:  
            case 11:  
                days = 30;  
                break;  
            case 2:  
                days = 28;  
                break;  
            default:  
                days = 31;  
                break;  
        }  
        return days;  
    }  
  
    @Override  
    public String toString() {  
        return String.format("y%dm%dd%d", year, month, day);  
    }  
  
    @Override  
    public boolean equals(Object other) {  
        if (other == null) return false;  
        if (this == other) return true;  
        if (!(other instanceof Date)) return false;  
        Date otherAsDate = (Date) other;  
        return this.day == otherAsDate.getDay() &&  
            this.month == otherAsDate.getMonth() &&  
            this.year == otherAsDate.getYear();  
    }  
}
```

```

@Override
public int compareTo(Date otherAsDate) {
    int diff = this.year - otherAsDate.getYear();
    if (diff != 0) return diff;
    diff = this.month - otherAsDate.getMonth();
    if (diff != 0) return diff;
    return this.day - otherAsDate.getDay();
}

public static class Builder {
    private final int year;
    public Builder(int year) {
        this.year = (year > 0) ? year : 1970;
    }
    public Date build(int day, int month) {
        if (month < 1 || month > 12 || day < 1 || day > daysPerMonth(month))
            return new Date(1, 1, year);
        return new Date(day, month, year);
    }
}
}

```

3. Classi `FormattedDate`, `FormattedDateConverter`, `ItalianDate`, `AmericanDate`

```

In [3]: abstract class FormattedDate extends Date {
    protected final String format;
    protected final String[] months;

    public FormattedDate(int day, int month, int year, String format, String[] months) {
        super(day, month, year);
        this.format = format;
        this.months = months;
    }

    public final String printFormat() { return format; }
    public final String getMonthAsString() { return months[getMonth()-1]; }
    public abstract String prettyPrint();
}

@FunctionalInterface
interface FormattedDateConverter {
    FormattedDate convert(FormattedDate date);
}

class ItalianDate extends FormattedDate {
    private static final String[] MONTHS_IT = { "gennaio", "febbraio", "marzo", "aprile", "maggio",
    "giugno", "luglio", "agosto", "settembre", "ottobre", "novembre", "dicembre" };
    public ItalianDate(int day, int month, int year) {
        super(day, month, year, "dd/mm/yyyy", MONTHS_IT);
    }
    @Override
    public String prettyPrint() { return day + " " + getMonthAsString() + " " + getYear(); }
    @Override
    public String toString() { return day + "/" + getMonth() + "/" + getYear(); }
}

class AmericanDate extends FormattedDate {
    private static final String[] MONTHS_US = { "January", "February", "March", "April", "May", "June",
    "July", "August", "September", "October", "November", "December" };
    public AmericanDate(int day, int month, int year) {
        super(day, month, year, "mm/dd/yyyy", MONTHS_US);
    }
    @Override
    public String prettyPrint() { return getMonthAsString() + " " + day + ", " + getYear(); }
    @Override
    public String toString() { return getMonth() + "/" + getDay() + "/" + getYear(); }
}

```

4. Tipi Generici: `Pair<F, S>`, `OrderedPair<T>`, `DatePair`, `DateInterval`

```
In [4]: class Pair<F, S> {
    private final F first;
    private final S second;

    public Pair(F first, S second) {
        this.first = first;
        this.second = second;
    }

    public F getFirst() { return first; }
    public S getSecond() { return second; }
}

class OrderedPair<T extends Comparable<T>> {
    private final T first;
    private final T second;

    public OrderedPair(T first, T second) {
        this.first = first;
        this.second = second;
        if (first.compareTo(second) > 0)
            System.out.println("Pair not ordered!"); // Pair not ordered!
    }

    public T getFirst() { return first; }
    public T getSecond() { return second; }
}

class DatePair extends Pair<Date, Date> {
    public DatePair(Date left, Date right) { super(left, right); }
    public Date getLeft() { return getFirst(); }
    public Date getRight() { return getSecond(); }
    @Override
    public String toString() {
        return String.format("[%s .. %s]", getLeft().toString(), getRight().toString());
    }
}

class DateInterval extends OrderedPair<Date> {
    public DateInterval(Date left, Date right) { super(left, right); }
    public Date getLeft() { return getFirst(); }
    public Date getRight() { return getSecond(); }
    @Override
    public String toString() {
        return String.format("[%s .. %s]", getLeft().toString(), getRight().toString());
    }
}
}
```

5. Classe `MainDate` ed Esecuzione

```
In [5]: class MainDate {
    public static void main(String[] args) {
        Date.Builder dateBuilder = new Date.Builder(2025);
        Date d1 = dateBuilder.build(10, 9);
        Date d2 = dateBuilder.build(10, -1);
        System.out.println(d1.toString()); // y2025m9d10
        System.out.println(d2.toString()); // y2025m1d1

        Time init = new Time() {
            @Override public int getHours() { return 0; }
            @Override public int getMinutes() { return 0; }
            @Override public int getSeconds() { return 0; }
            @Override public String toString() {
                return String.format("%02d:%02d:%02d", getHours(), getMinutes(), getSeconds());
            }
        };
        System.out.println(init.toString()); // 00:00:00

        FormattedDateConverter toAmerican =
            d -> new AmericanDate(d.getDay(), d.getMonth(), d.getYear());
        System.out.println(toAmerican.convert(new ItalianDate(11, 11, 2025)) instanceof AmericanDate); //
true
}
```

```

Pair<String, FormattedDate> event = new Pair<>("00P exam", new ItalianDate(2, 2, 2026));
System.out.println(event.getFirst() + " on " + event.getSecond().prettyPrint()); // 00P exam on 2
febbraio 2026

DatePair dp = new DatePair(new Date(1, 2, 2025), new Date(1, 1, 2025));
System.out.println("date pair: " + dp.toString()); // date pair: [y2025m2d1 .. y2025m1d1]

DateInterval di = new DateInterval(new Date(1, 2, 2025), new Date(1, 1, 2025)); // costruttore
OrderedPair stampa: Pair not ordered!

Pair unknown = new Pair("00P exam", new ItalianDate(2, 2, 2026));
System.out.println(unknown.getFirst().toString() + " on " + ((ItalianDate)
unknown.getSecond()).prettyPrint()); // 00P exam on 2 febbraio 2026
    }
}
MainDate.main(null);

```

```

y2025m9d10
y2025m1d1
00:00:00
true
00P exam on 2 febbraio 2026
date pair: [y2025m2d1 .. y2025m1d1]
Pair not ordered!
00P exam on 2 febbraio 2026

```